CSS Grid Portfolio

A study guide: build the 6-card project grid, understand every property, and learn the concepts underneath it.

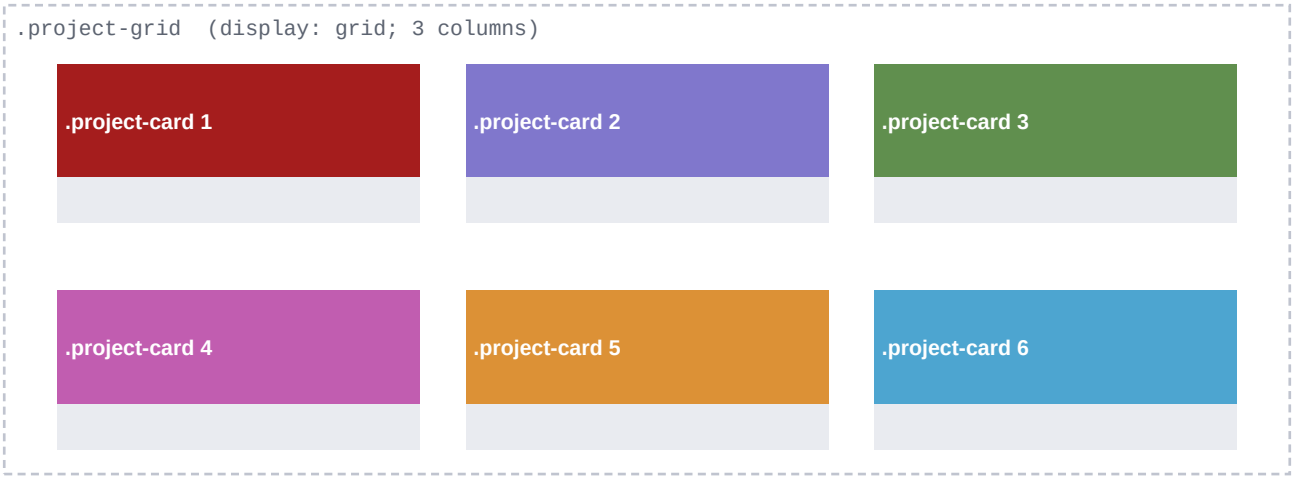
How to use this guide
Do not read it top to bottom once. Read Part 1 and Part 2, then **type the code yourself** in a fresh folder, then use Part 3 to look up whatever you do not understand. Part 6 has questions. If you cannot answer them without scrolling up, you have not learned it yet.

Contents

- Part 1 - The big picture: what you are building and the 4 ideas that make it work
- Part 2 - HTML structure and the class-name confusion, resolved
- Part 3 - Core concepts: selectors, box model, Grid, Flexbox, responsive design
- Part 4 - Property reference tables (every property from the chat)
- Part 5 - Complete working code
- Part 6 - Semantic HTML: main, article, section, header, footer, nav, aside
- Part 7 - Mistakes beginners make, plus practice questions with answers
- Part 8 - Concepts worth learning next (and in what order)

Part 1 - The big picture

The design is a heading followed by six cards in three columns and two rows. Each card has a coloured image area on top and a white info area below.



The single most important insight

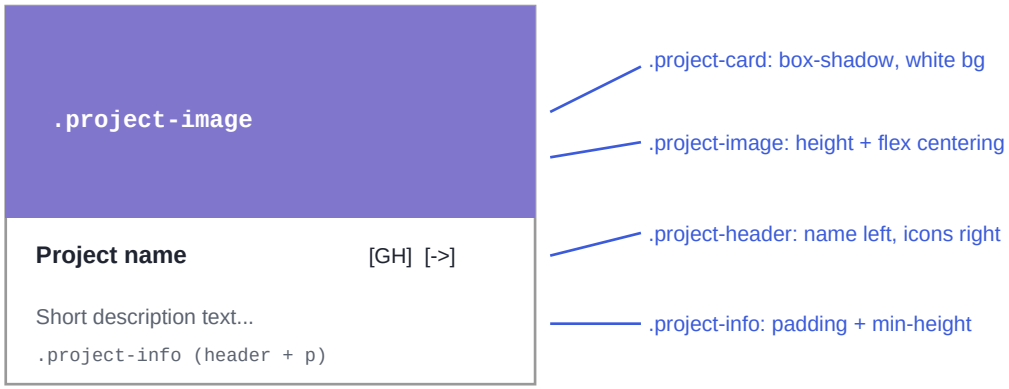
Do **not** think of six separately positioned boxes. Build **one reusable card**, repeat it six times, and let the container arrange them. That is what separates a layout that scales from one that breaks when you add a seventh card.

The four ideas that make the whole design work

Idea	Job in this design	Key properties
HTML structure	Nested containers give CSS things to grab	`article`, `div`, class names
CSS Grid	Arranges the 6 cards into 3 columns x 2 rows	`display: grid`, `grid-template-columns`, `gap`
Flexbox	Aligns things *inside* a card (title left, icons right; centred text)	`display: flex`, `justify-content`, `align-items`
Media queries	3 columns on desktop, 2 on tablet, 1 on phone	`@media (max-width: ...)`

Rule of thumb: Grid vs Flexbox
Grid = two dimensions (rows *and* columns): page layouts, card galleries. **Flexbox** = one dimension (a row *or* a column): navbars, aligning items inside a component. Real pages use both, exactly like this design: Grid outside, Flexbox inside.

Anatomy of one card



Part 2 - HTML structure and class names

Your confusion about `.project-grid`, `.project-header` and `.project-links` has one cause: **class names are not CSS keywords.** You invent them in HTML, then reuse the exact same name in CSS to select the element. Rename `project-grid` to `my-box` in both files and everything still works.

One complete card (the other five are identical)

```
<div class="project-grid">                    <!-- container for ALL cards -->

  <article class="project-card">              <!-- ONE card -->

    <div class="project-image red">           <!-- coloured top area -->
      screenshot<br>of project
    </div>

    <div class="project-info">                <!-- white bottom area -->

      <div class="project-header">            <!-- title + icons row -->
        <h2>Project name</h2>

        <div class="project-links">           <!-- the two icons -->
          <a href="#">GitHub</a>
          <a href="#">Open project</a>
        </div>
      </div>

      <p>Short description of the project.</p>

    </div>
  </article>

  <!-- ...5 more <article class="project-card"> here... -->
</div>
```

Which element has which class

Class	Element	Its job	CSS that targets it
`project-grid`	`<div>`	Holds all six cards; becomes the grid	`.project-grid { display: grid }`
`project-card`	`<article>`	One whole card (shadow, white background)	`.project-card { box-shadow }`
`project-image`	`<div>`	Coloured top area, centres its text	`.project-image { display:flex }`
`project-info`	`<div>`	White bottom area, wraps header + text	`.project-info { padding }`
`project-header`	`<div>`	Puts title and icons on one row	`.project-header { display:flex }`
`project-links`	`<div>`	Holds the two icons, spaces them	`.project-links { display:flex }`

How HTML nesting maps to a tree

```
.project-grid
 +-- .project-card            (x6)
      +-- .project-image
      +-- .project-info
           +-- .project-header
           |    +-- h2
           |    +-- .project-links
           |         +-- a
           |         +-- a
           +-- p
```

Critical consequence: `display: grid` on `.project-grid` controls only its **direct children** (the cards). It does nothing to the elements inside each card. That is why `.project-header` needs its own `display: flex`. Every layout container controls its direct children only.

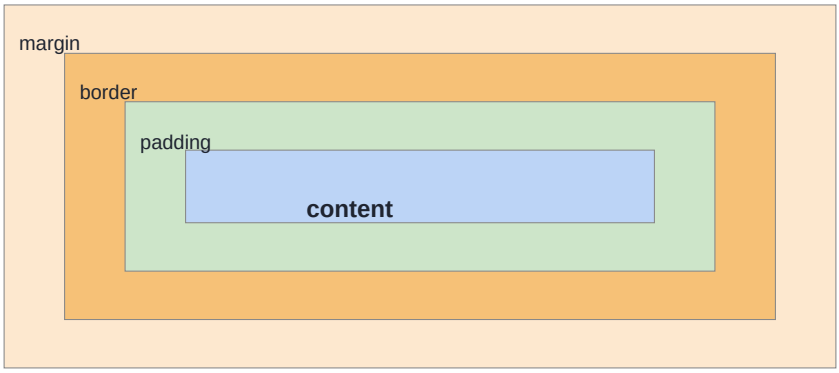
Part 3 - Core concepts

1. Selectors: how CSS finds elements

Selector	Meaning
`h2`	Every h2 on the page (tag selector, no dot)
`.project-card`	Any element whose class is project-card (dot = class)
`.project-header h2`	Any h2 **inside** .project-header (a space means descendant)
`.project-links a`	Any a inside .project-links
`*`	Every element (used for the box-sizing reset)

Classes are reusable: give six cards the same class and one CSS rule styles all six. Two classes on one element, like `class="project-image red"`, combine: shared styles from the first, the colour from the second. That is how six cards share one layout but have different colours.

2. The box model



Every element is a box: content, padding, border, margin. The default `box-sizing: content-box` makes `width` apply to the content *only*, so padding and border make the box wider than you wrote:

```
.card { width: 300px; padding: 20px; border: 5px solid black; }
/* content-box: 300 + 20 + 20 + 5 + 5 = 350px total (surprise!) */
/* border-box : 300px total. Padding and border fit INSIDE the width. */

* { box-sizing: border-box; }   /* the standard reset: put this at the top of every project */
```

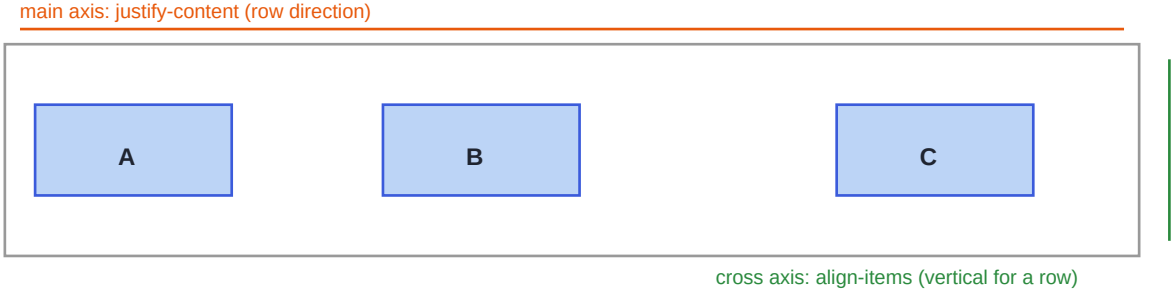
padding = space inside the border (background shows through). **margin** = space outside (transparent). **gap** = space between children, set on the parent (cleaner than margins on every card).

3. CSS Grid

```
.project-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 58px 46px;          /* row-gap  column-gap */
}
```

- `repeat(3, 1fr)` is shorthand for `1fr 1fr 1fr`: three equal columns.
- **fr** = a fraction of the leftover space. `2fr 1fr 1fr` gives the first column twice the share of the others.
- `minmax(0, 1fr)` lets columns shrink below their content's natural width, which prevents long text or images from forcing overflow. For simple layouts `repeat(3, 1fr)` is fine.
- You never place cards manually. Grid auto-flows 6 children into 3 + 3. Add a 7th card and a third row appears with no CSS change.
- `gap: 58px 46px`: **first value = rows (vertical), second = columns (horizontal)**. One value applies to both.

4. Flexbox




```
.project-header {
  display: flex;                    /* children sit in a row */
  justify-content: space-between;   /* MAIN axis: title far left, icons far right */
  align-items: center;              /* CROSS axis: vertically centred */
  gap: 10px;
```

```
}
```

- Two axes. For the default `flex-direction: row`: main axis = horizontal, cross axis = vertical.
- `justify-content` = main axis. `align-items` = cross axis. **If you set `flex-direction: column` the axes swap.** This is the most common Flexbox confusion.
- Centring anything perfectly = `display:flex; justify-content:center; align-items:center`.
- `flex-shrink: 0`: by default flex items shrink when space runs out. 0 means do not shrink (icons stay legible).

5. Responsive design: media queries

```
.project-grid { grid-template-columns: repeat(3, minmax(0, 1fr)); }   /* default: desktop */

@media (max-width: 1000px) {                /* viewport 1000px or narrower */
  .project-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); }
}
@media (max-width: 600px) {                 /* viewport 600px or narrower */
  .project-grid { grid-template-columns: 1fr; }
}
```

- The same HTML adapts: you change the **column count**, not the markup.
- **Order matters.** Later rules override earlier ones of equal strength. That is why the 600px query sits *after* the 1000px one: on a phone both match, and the later one wins.
- Queries measure the **viewport** (browser window width), not the physical device.
- The line `<meta name="viewport" content="width=device-width, initial-scale=1.0">` in the HTML head is required, or phones pretend to be ~980px wide and ignore your queries.

Part 4 - Property reference

Look things up here. Do not memorise. Learn the *problem* each property solves.

Layout

Property	What it does	Used for
`display: grid`	Makes element a grid container; its children become grid items	Arranging the 6 cards
`grid-template-columns`	Defines the columns and their sizes	3 / 2 / 1 columns
`gap / row-gap / column-gap`	Space *between* grid/flex items	Space between cards
`display: flex`	Makes element a flex container (row by default)	Header, icons, centring
`justify-content`	Distributes items along the main axis. Values: flex-start, center, flex-end, space-between, space-around, space-evenly	Title left, icons right
`align-items`	Aligns items on the cross axis	Vertical centring
`flex-direction`	row (default) or column; swaps the axes	Stack on narrow screens
`flex-shrink`	Whether an item may shrink when space is short (0 = no)	Protect the icons

Sizing and spacing

Property	What it does	Used for
`box-sizing: border-box`	Width/height include padding and border	Predictable sizes everywhere
`min-width: 0`	Lets a grid/flex item shrink below its content width	Stops overflow in cards
`max-width`	Upper limit on width	Stop page being huge on wide monitors
`margin: 0 auto`	0 top/bottom, auto left/right: splits spare space equally	Centring a block container
`min-height`	Minimum height; element may grow	Cards with varying text
`height`	Fixed height; content may overflow	Image area (fixed on purpose)
`padding`	Space inside the border	Breathing room in .project-info

Visual style

Property	What it does	Used for
`box-shadow`	**offset-x offset-y blur colour**. Example: 0 5px 10px rgba(0,0,0,.22) = no sideways shift, 5px down, soft blur, 22% black	Lifts white cards off white page
`rgba(r,g,b,a)`	Colour with transparency. a: 0 invisible to 1 solid	Soft shadows
`font-weight: normal`	Removes default bold from headings (normal = 400, bold = 700)	Light-looking headings
`line-height: 1.35`	Line spacing. A unitless number scales with font-size (46px x 1.35 = 62px)	Readable two-line text
`text-align: center`	Centres text inside its lines. Different from flex centring, which moves the whole box	Centring both lines relative to each other
`object-fit: cover`	Image fills its box and crops overflow. contain = show whole image with gaps. fill = stretch (distorts)	Real screenshots in the image area
`text-decoration: none`	Removes underline from links	Clean icon links

Part 5 - Complete working code

Save as `index.html` and `style.css` in the same folder. Open index.html in a browser. Only one card is shown; copy the block five times and change the colour class (red, purple, green, pink, orange, blue).

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Work</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <main class="portfolio">
    <h1>My work</h1>
    <div class="project-grid">

      <article class="project-card">
        <div class="project-image red">screenshot<br>of project</div>
        <div class="project-info">
          <div class="project-header">
            <h2>Project name</h2>
            <div class="project-links">
              <a href="#" aria-label="GitHub repository">GH</a>
              <a href="#" aria-label="Open project">&#8599;</a>
            </div>
          </div>
          <p>Short description of the project.</p>
        </div>
      </article>

      <!-- copy the article 5 more times, change red -> purple, green, ... -->
    </div>
  </main>
</body>
</html>
```

style.css

```
* { box-sizing: border-box; }

body { margin: 0; background: white; color: #171717; font-family: Georgia, serif; }

.portfolio { padding: 50px 4%; max-width: 1600px; margin: 0 auto; }

h1 { font-size: 44px; font-weight: normal; margin: 0 0 12px; }

.project-grid {
  display: grid;
  grid-template-columns: repeat(3, minmax(0, 1fr));
  gap: 58px 46px;
}

.project-card { background: white; box-shadow: 0 5px 10px rgba(0,0,0,0.22); min-width: 0; }

.project-image {
  height: 278px;
  display: flex; align-items: center; justify-content: center;
  text-align: center; color: white; font-size: 46px; line-height: 1.35;
}
.red    { background: #a51d1d; }   .purple { background: #8077cc; }
.green  { background: #608f4e; }   .pink   { background: #c15db0; }
.orange { background: #dc9136; }   .blue   { background: #4da5d0; }

.project-info { padding: 14px 22px 24px; min-height: 182px; }

.project-header {
  display: flex; align-items: center; justify-content: space-between; gap: 10px;
}
.project-header h2 { font-size: 24px; font-weight: normal; margin: 0; }

.project-links { display: flex; align-items: center; gap: 18px; flex-shrink: 0; }
.project-links a {
  color: #171717; font-family: Arial, sans-serif;
  font-size: 25px; font-weight: bold; text-decoration: none;
}

.project-info p {
  font-family: Arial, sans-serif; font-size: 17px; line-height: 1.3;
  color: #555; margin: 16px 0 0;
}

@media (max-width: 1000px) {
  .project-grid { grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 32px; }
  .project-image { height: 240px; font-size: 36px; }
}
@media (max-width: 600px) {
  .portfolio { padding: 24px 20px; }
  h1 { font-size: 36px; }
  .project-grid { grid-template-columns: 1fr; gap: 28px; }
}
```

To make it real: replace the colour divs with `<img src="shot.png" alt="...">`, then style with `.project-image img { width:100%; height:278px; object-fit:cover; }`, and swap the text icons for SVG icons.

Part 6 - Semantic HTML

A `<div>` says nothing about its content. Semantic elements say **what the content is**, which helps screen readers, search engines, and you reading your own code in six months. They look identical to a div until you add CSS.

The rule: choose the element by what the content *means*. Control looks with CSS. Never pick `<article>` just because you want a card-shaped box.

Element	Meaning	Portfolio example
`<header>`	Introductory content: logo, title, nav. Can belong to the page *or* to a single article/section	Site title + nav
`<nav>`	A group of major navigation links	About / Work / Contact links
`<main>`	The primary content of the page. Only **one** visible per page	Wraps all sections
`<section>`	A themed group of content, normally with a heading	"My work" section
`<article>`	Self-contained content that makes sense alone (blog post, news item, distinct project)	Each project card
`<aside>`	Related but secondary content: sidebar, side notes	"Technologies I use" box
`<footer>`	Closing info: copyright, contact. Can also belong to an article/section	© and email
`<div>`	Generic block container, no meaning	project-grid, project-header
`<span>`	Generic *inline* container (style part of a line of text)	Highlight one word

A sensible full-page skeleton

```
<body>
  <header>
    <h1>My Portfolio</h1>
    <nav> <a href="#about">About</a> <a href="#work">My Work</a> </nav>
  </header>

  <main>
    <section id="about"> <h2>About Me</h2> ... </section>

    <section id="work">
      <h2>My Work</h2>
      <div class="project-grid">
        <article class="project-card"> <h3>Weather App</h3> ... </article>
        <article class="project-card"> <h3>Calculator</h3> ... </article>
      </div>
    </section>
  </main>

  <footer> <p>&copy; 2026 My Portfolio</p> </footer>
</body>
```

Section vs article
If you can lift the content out and it still makes sense on its own (a project, a post), it is an **article**. If it is a chapter of the page grouped by topic (About, Work, Contact), it is a **section**.

Part 7 - Common mistakes, then practice

Mistakes that cost beginners hours

Mistake	Why it fails	Fix
Putting `justify-content` on the wrong element	It only affects the **direct children** of a flex container	Apply flex to the immediate parent
Forgetting the dot: `project-grid { }`	Without a dot CSS looks for a tag	Class = `.project-grid`
Typo between HTML and CSS class names	Names must match exactly; CSS fails silently	Inspect the element in browser DevTools
Expecting `display: grid` on a card to arrange all cards	Grid affects its own children, not siblings	Put grid on the parent container
Fixed `height` on a box with text	Text overflows when content is longer	Use `min-height` where content varies
Mixing up the two axes	Axes swap when flex-direction is column	Say the direction first, then pick the property
Media queries in the wrong order	Equal-strength rules: the last one wins	Large screens first, then smaller max-width queries in order
Using margins to space grid items	Edge items get unwanted outer margins	Use `gap` on the container

Practice questions

- **Q1.** Write the CSS to make a grid with 4 equal columns and 20px between everything.
- **Q2.** The title and icons sit on one row but the icons stick to the title instead of the far right. Which property is missing, and on which element?
- **Q3.** A card is `width:300px; padding:20px; border:5px solid` with the default box-sizing. What is its total width? And with border-box?
- **Q4.** What does `.project-links a` select? What would `.project-links.a` select (no space)?
- **Q5.** You swap to `flex-direction: column`. Which property now centres items horizontally?
- **Q6.** Why is `min-height` safer than `height` for the description area?
- **Q7.** Where in the HTML does the `<main>` element go, and why use `<article>` for a project card?
- **Q8.** Two media queries (max-width 1000px and 600px) both match on a 500px phone. Which wins and why?

Answers

- **A1.** `display:grid; grid-template-columns: repeat(4, 1fr); gap: 20px;`
- **A2.** `justify-content: space-between` on the flex container (`.project-header`). Without it, items bunch at the start.
- **A3.** content-box: 300+20+20+5+5 = **350px**. border-box: **300px**.
- **A4.** Every `<a>` *inside* an element with class project-links. `.project-links.a` would select one element that has *both* classes project-links and a (almost never what you want).
- **A5.** `align-items` (the cross axis is now horizontal).

- **A6.** `min-height` lets the box grow with longer text; fixed `height` lets text spill out.
- **A7.** `<main>` wraps the page's primary content inside body (one per page). A project is a self-contained item, so `<article>` describes it correctly.
- **A8.** The 600px one, because it comes later in the file and both have equal specificity.

Part 8 - Concepts worth learning next

These are the ideas behind the code that will make you faster. Roughly in the order to learn them.

Concept	Why it matters	How to start
Browser DevTools (F12)	Click any element, see which CSS applies, edit values live. The fastest way to learn and debug CSS	Right-click the grid, Inspect, toggle properties on/off
Cascade and specificity	Decides which rule wins when several target the same element: inline > id > class > tag; then later beats earlier	Make two conflicting rules and watch which one DevTools crosses out
Inheritance	Some properties (color, font-family) pass down to children; others (border, padding) do not	Set color on body, see headings follow
Units: px, %, rem, em, fr, vw	px is fixed; rem scales with the root font size (better for accessibility); % is relative to the parent	Replace px font sizes with rem and change the browser font size
Mobile-first CSS	Write the phone layout first, then use `min-width` queries to add columns for bigger screens. Usually cleaner than the desktop-first approach used here	Rewrite this grid with 1 column default and min-width queries
Auto-fit grids	`repeat(auto-fit, minmax(280px, 1fr))` gives a responsive grid with **no media queries at all**	Try it and resize the window
CSS variables	`--accent: #3b5bdb;` then `color: var(--accent)`. Change a colour in one place	Move the 6 card colours into variables
Accessibility basics	alt text on images, `aria-label` on icon-only links (already in the code), sufficient colour contrast	Check white text on the orange card; contrast is weak

Suggested order of study

- **Week 1:** box model, selectors, DevTools. Rebuild one card from scratch without looking.
- **Week 2:** Flexbox (navbars, centring, card headers). Do ten small exercises, not one big project.
- **Week 3:** Grid and media queries. Rebuild this entire page, then add a 7th card to see it reflow.
- **Week 4:** Put your real Android and web projects in the cards with real screenshots, then deploy it.

The honest advice
Copying working code teaches you almost nothing. The learning happens when you **break it on purpose**: change `space-between` to `center`, change `3` to `4` in the repeat, delete `box-sizing`, and watch what moves. Ten minutes of breaking this page beats an hour of rereading this PDF.